



**UNIVERSITY *of*
GREENWICH**

UNDERGRADUATE PROJECT REPORT

V - The AI Assistant

Luu Truong The Quyen

Bachelor of Science with Honours in Computing

ID: GCS220316

SUBMISSION DATE: August 1st, 2026

V - The AI Assistant

Abstract

Autonomous AI agents possess endless possibilities but constantly fail in production environments due to the inherent probabilistic nature of LLMs, large language models. Standard ReAct (Reason + Act) structures are highly susceptible to compounding errors, in which a single misstep in a multi-stage workflow can snowball into systemic failure. In addition, monolithic context windows lead to “context rot,” and naive retrieval pipelines distract the model’s focus; therefore, the agent sounds confident but outputs nonsense.

Project V addresses and evaluates these critical weaknesses by changing from a fragile single-agent model to a deterministic, compound AI system. V is an architecture with DDD (domain-driven design), where it isolates functions into specialized sub-agents and restricted tool registries

To address context limitations, the project utilizes progressive disclosure alongside a reversible memory compaction engine, closing the gap between volatile short-term RAM and an SQLite ROM. Furthermore, safe automation is guaranteed by a hard-bounced sandbox and event-driven human-in-the-loop gates for destructive actions.

Ultimately, V showcases that AI reliability is not achieved by simply refining models, but by crafting a robust, observable microservice-like intelligence layer wrapped around them to ensure safe and deterministic task execution

Acknowledgements

Table of Contents

1. Introduction.....	6
1.1. Project Introduction.....	6
1.2. Project Objectives.....	7
1.3. Project Plan.....	8
1.4. Project Outcomes.....	11
1.5. Project Evaluation.....	11
2. Literature Review.....	12
2.1. The Compounding Error Problem in ReAct Structures.....	12
2.2. Mitigating Context Rot via Progressive Disclosure.....	13
2.3. Compound AI Systems vs. Monolithic Agents.....	14
3. Technology and Tools.....	16
4. Software Product Requirements.....	18
4.1. Overview of Similar Products.....	18
4.2. User Stories and Use Case Diagram.....	19
4.3. System Flow and Specifications.....	21
4.3.1. Use Case Specifications.....	21
4.3.2. Activity Diagram.....	22
4.4. Entity Relationship Diagram (ERD).....	23
4.5. Sitemap.....	23
5. Software Development Methodologies.....	24
5.1. Waterfall Model.....	24
5.2. Spiral Model.....	24
5.3. Rapid Application Development (RAD / Prototyping).....	24
5.4. Agile Methodology.....	24
5.5. Selection of Methodology and Justification.....	24
6. Design and Implementation.....	24
6.1. Product Analysis and Design.....	24
6.1.1. System Interface Overview.....	24
6.1.2. GUI Design Process.....	24
6.1.3. Product Analysis.....	24
6.1.4. Architecture Flow Design.....	24
6.2. Graphical Interface Illustrations.....	24
6.2.1. AI Sentinel Homepage.....	24
6.2.2. AI Sentinel Exploratory Data Analysis.....	24
6.2.3. AI Sentinel Uni-Modal Result.....	24
6.2.4. AI Sentinel Multi-Modal Result.....	24
6.2.5. AI Sentinel Interactive.....	24
6.3. Product Implementation.....	25
6.3.1. Data Collection.....	25
6.3.2. Exploratory Data Analysis (EDA).....	25
6.3.3. Experiments and Results.....	25
6.4. Product Evaluation.....	25
6.4.1. Strengths.....	25

6.4.2. Limitations and Challenges.....	25
7. Conclusion.....	25
7.1. Knowledge, Experience, and Insights.....	25
7.2. Achievement Assessment.....	25
7.3. Future Works.....	25
8. References.....	25
9. Appendix.....	25
10. Declaration of AI Use.....	25

List of Figures

Tabular Directory

1. Introduction

1.1. Project Introduction

The ever-evolving technology field is rapidly transitioning from stateless LLM chatbots (Stage 1) and basic Retrieval-Augmented Generation (RAG) systems (Stage 2) to autonomous AI agents with endless capabilities, including complex, multi-step operations (Stage 3). While the development is promising, the technology itself is advancing toward a hard operational wall. Meaning, common industry forecasts that 40% of generative AI projects will be discarded by 2027 because the foundation is fundamentally unprepared for production environments (Paolo Perrone, 2026). These systems are explicitly designed for the “happy path” cases and scenarios and break down when they encounter messy realities of edge cases or simply version updates.

At present, the common approach to agentic AI relies on a poorly comprehensive assumption that providing a massive, smart language model with a detailed prompt and unfettered tool access, trusting it to reason its way to a desirable outcome dynamically. This methodology is fragile since the nature of non-deterministic systems fails quietly and confidently (Paolo Perrone, 2026). The problem is calculated to be the compounding error dilemma. If an agent has an 85% success rate per individual step, a simple five-step flow has less than a 45% chance of succeeding since, at every step in the flow, a small error cascades into the next action. A traditional chatbot hallucinating a response is merely a UX failure, whereas an autonomous agent hallucinating a production command, namely executing a database reset, is a catastrophic systemic failure (Zaid Mohd, 2026).

Taking the above into consideration, Project V rejects monolithic, unconstrained AI in favour of a Compound AI System architecture (Stage 4). By viewing the AI model not as an omnipotent brain, but as a probabilistic reasoning engine contained within a strict, deterministic framework. In that way, V could achieve verifiable robustness.

Shifting from relying on the model’s unguided judgments, V enforce precision through crafted engineering mechanisms:

- **Graph-Based State Machine:** For strict orchestration and task delegation, making sure that the Agent cannot drift beyond its assigned domain.
- **Progressive Context Disclosure:** Feeding the model only the exact data it needs at the moment it needs it, minimizing “context rot” condition and attention dilution.
- **Hard-Bounded Sandbox Execution:** setting strict computational and inferential sensors that separate code execution and require mandatory human-in-the-loop authorization for any destructive actions.

1.2. Project Objectives

Based on the challenges identified in creating an autonomous desktop assistant, this project focuses on the research and implementation of a better, tool-augmented AI agent with the following core ideas:

- 1. Balanced Context with Two-Tier Memory:** Configure & tune persistent memory across two data ingestion streams - a short-term writable RAM view buffer and a long-term compressed SQLite transaction log. The decision is motivated by ensuring the agent can reach peak achievable conversation precision through balancing strengths of each memory processing pipeline. Maintaining distance from the “context rot” trap.
- 2. Robust Tool Routing and Dynamic Task Management:** Create a strict routing core that is designed only to provide highly accurate execution of tools, alongside dynamic calculation of task urgency and temporal deadlines. This objective aims to resolve “when” and “how” the AI agent needs to interact with its system ledger or the web without the user’s intervention.
- 3. Optimized Asynchronous Inference Efficiency:** Making sure that all the processing and inference time for the dual-LLM workflow remains responsive when active. This goal is vital for maintaining practical feasibility, seamless streaming and a natural UX experience.
- 4. Advanced Against LLM Hallucinations:** Intent to achieve max reliability by restricting strict structural guardrails, namely as Security Tiers and explicit routing gates. Therefore, providing consistently accurate state changes and eliminating “Lazy LLM” conditions; hence, the agent never makes up success outputs without physical tool executions.
- 5. Seamless Unified Agentic Workflow:** Allow users to do anything from open-ended philosophical discussions to scraping the web to rigid database management without breaking away from your workflow.
- 6. Deployment of a Synchronized Desktop Interface:** Create an interface that reacts quickly. Meaning to craft an experience that brings users into one window where they can directly interact with all of V's features. Additionally,

this also means making sure that things that users see (dynamic task lists, for example) are completely in sync with backend processes.

1.3. Project Plan

Timeline (7 months total)	Tasks conduct
Month 1: Research and Data Collection	
Week 1:	Define specific multi-task objectives, success metrics, and hardware/software constraints.
Week 2:	Conduct a literature review on relevant Multi-Task Learning (MTL) architectures and routing mechanisms.
Week 3:	Identify, scrape, or acquire the necessary datasets for all targeted tasks.
Week 4:	Perform Exploratory Data Analysis (EDA) and finalize the data storage schema and backend environment.
Month 2: Data Preprocessing	
Week 5:	Clean and normalize the raw data; programmatically handle missing values and outliers.
Week 6:	Apply tokenization, structural formatting, and task-specific labelling to the datasets
Week 7:	Build automated data pipelines and unified data loaders to efficiently handle multi-task inputs.
Week 8:	Perform train/validation/test splits and conduct a baseline data quality validation pass.
Month 3: Model Architecture Development	
Week 9:	Draft and test baseline single-task models to establish performance benchmarks.
Week 10:	Design and code the shared representation layers (e.g., the shared encoder or backbone).
Week 11:	Design the task-specific heads (e.g., classification, generation, routing mechanisms).

Week 12:	Implement the preliminary multi-task architecture in your chosen framework (e.g., PyTorch).
Week 13:	Debug model compilation, verify gradient flow, and successfully test preliminary forward passes.
Month 4: Model Architecture Development	
Week 14:	Implement dynamic routing, semantic gates, or task-weighting mechanisms to balance objectives.
Week 15:	Integrate optimization strategies to mitigate negative transfer between conflicting tasks.
Week 16:	Refine structural guardrails and output formatting constraints for the generation heads.
Week 17:	Conduct a rigorous final code review and freeze the core model architecture.
Month 5: Model Training and Multi-Task Learning	
Week 18:	Set up cloud infrastructure, allocate GPUs, and initialize logging trackers (e.g., Weights & Biases).
Week 19:	Conduct initial hyperparameter tuning on a small, representative subset of the data.
Week 20:	Launch full-scale training on the primary shared layers and monitor initial convergence.
Week 21:	Analyze training curves in real-time, diagnose loss plateaus, and adjust learning rate schedulers if necessary.
Month 6: Model Training and Multi-Task Learning	
Week 22:	Unfreeze task-specific heads and begin fine-tuning for individual task objectives.
Week 23:	Implement regularization techniques and actively address any signs of overfitting or catastrophic forgetting.
Week 24:	Conduct ablation studies on specific architecture components to validate their contribution to performance.
Week 25:	Perform comprehensive validation testing across all task metrics against the Week 9 baselines.
Week 26:	Freeze all final model weights and export the model state for

	inference optimization.
Month 7: Finalization and Submission	
Week 27:	Integrate the trained model with the deployment backend (e.g., Uvicorn/FastAPI) and frontend UI.
Week 28:	Conduct end-to-end system testing, focusing on latency, robustness, and load balancing.
Week 29:	Draft the formal technical documentation, compile the architecture diagrams, and write user manuals.
Week 30:	Perform final repository cleanup, prepare the project presentation, and submit the final deliverables.

Table 1: Project Plan

1.4. Project Outcomes

Following a stiff phase of architectural design and interactive refinement, the V agent has achieved milestones that transformed basic LLM interactions. The final results of this project build a bridge to close the gap between theoretical agentic models and a production-grade, highly viable desktop application.

- 1. Dynamic Context Compaction:** The system successfully created a compaction engine to resolve context rot. It works by condensing large conversation histories into high-fidelity representations. Meanwhile, it also preserves active constraints (Jim Allen Wallace, 2026). This mechanism effectively combines the best of both the short-term RAM window and long-term SQLite ROM.
- 2. Secure Sandbox Execution Management:** This architecture also replaced unbounded workspace risks with restricted file system execution boundaries. The system required human-in-the-loop gates before the agent can touch any destructive tools (Molisha Shah, 2026).
- 3. Resilient Graph-Based Orchestration:** The spine transitioned from a rigid linear execution flow to a loop-based orchestration layer. This state machine audits expectations against actual tool observations (Paolo Perrone, 2026). Therefore, V actively mitigates the compounding error problem prevalent in multi-step workflows.
- 4. Robust Dual-Database Architecture:** The project integrates ChromaDB for semantic search, combined with an SQLite relational database to ensure complex data integrity. In addition, V utilizes two-tiered guardrails - a fast similarity check followed by factual validation - to avoid naive RAG hallucinations (Birgitta Böckeler, Chris Ford, 2026).

1.5. Project Evaluation

The development of the V agent leads to the practical conclusion that, in order to use trustworthy AI, the system around the AI should be engineered with the same rigour as the LLM that powers the AI. The project proves that the robustness of an AI agent depends on the system in which it operates by showing that the move from fragile and inflexible linear pipelines to a flexible and resilient event-based architecture was necessary for handling state and API changes seamlessly.

Interaction with the architecture presented in our literature review showed an unexpected result in prompt design: limiting the tools that an agent can use and applying progressive disclosure actually increase its focus and the quality of its output (MindStudio, 2026). This is achieved by using computational sensors (like `ram_window.py`) to detect the limits of buffers and keep standards and situational awareness without any human involvement (Birgitta Böckeler, Chris Ford, 2026).

Future research will be directed towards moving the agent from correlational pattern matching to causal tool models-based reasoning. Being able to understand causations of its actions (for example, knowing that an authentication error is caused by not having a token created in the previous step) will significantly increase its ability to detect errors and recover from them autonomously. Expansion of the multi-agent intelligence layer for specialized tasks is another promising direction for real-world applications (Zaid Mohd, 2026).

2. Literature Review

2.1. The Compounding Error Problem in ReAct Structures

The ReAct (Reason + Act + Observe) loops are the basis of the functional operation of contemporary AI agents. At the same time, the practical application of these models encounters numerous problems because of accumulating errors in production. First of all, the problem lies in the design of the ReAct loop since it relies on sequential probability instead of deterministic causality. It is proven by error propagation calculations that the use of autonomous workflows does not help avoid the issue of errors but makes it even more severe. The empirical analysis of success rates of workflows proves that the agent with approximate accuracy of 85% per step achieves a success rate of about 44% for a five-step workflow. In the case of a ten-step workflow, almost eighty percent of users fail to perform it. The analysis of the cases where the ReAct agent fails shows that the early mistake is not an exception: the agent usually deviates enough to fail by the fifth-sixth step of a twenty-step task.

The inability to reason and the probabilistic blind spot lie at the core of ReAct models. These are probabilistic models which try to predict the most probable next action according to some pattern observed in the training data instead of developing a causal model of the tool-environment. Inability to conduct causal reasoning leads to the failure of detecting an error: if some step was performed incorrectly, the agent would not detect the missing expected change and go to the next step.

In practice, the silent confident failure can lead not only to user interface issues but also to catastrophic failures. There is an example of one such incident from July 2025 when the agent was supposed to perform routine maintenance while there was a code freeze (Paolo Perrone, 2026). Making a chain of individual plausibility of probabilistic decision-making, the agent made a decision that caused a breach in the production database. As a result, lacking any checkpoints to interrupt the chain of errors, the agent created four thousand fictional user accounts and falsified the system log.

To mitigate architectural flaws, bounded operation constraints should be introduced. To make the ReAct architecture functional in the real-life environment, strict checkpoints and human-in-the-loop permission gates should be implemented before irreversible actions.

2.2. Mitigating Context Rot via Progressive Disclosure

One such common intuition for building an AI agent is pre-loading all possible reference files, style guides, and API schemas into the context window before executing a task. While this might seem like a good way to increase the efficiency of your agent at first glance, it is one of the best ways to degrade output quality and induce a process known as "context rot." The more filled the working window is with material, the lower the attention of the model is going to be. Instructions that appear later in the sequence, such as on token 50,000, get significantly less attention compared to those earlier, say token 500. The signal-to-noise ratio drops sharply because there is too much competition for the attention of the model among irrelevant reference materials, increasing the risk of contradiction.

To avoid this issue, modern agent architectures use a technique called progressive disclosure (MindStudio, 2026), which has been borrowed from User Experience (UX) engineering. Progressive disclosure in AI involves loading only the reference material needed to accomplish a certain task step into the context window. It is not just about concise prompting. It is a technique to handle context dynamically. Initially, the agent starts with a bare-minimum working context consisting of only the task definition, the current state of the task and an index of all available references. Then the agent loads only the required material at certain task steps and removes it afterwards once the step is complete.

There are several key benefits to this approach:

- **Sharp focus:** The model works with only the relevant information, instead of paying attention to multiple unnecessary reference files.

- **Instruction clarity:** Critical behaviour instructions and output constraints stay relevant to the total amount of the context, making sure that they won't be hidden away under a pile of documentation.
- **Diagnosability:** In case there is an error in a progressively disclosed workflow, developers can pinpoint the context load that caused the error, unlike monolithic contexts, which are harder to analyze for errors.
- **Efficiency:** There is much lower token overhead because the system does not allocate tokens that do not add value to the reasoning.

In summary, these examples show that context engineering, i.e., careful control over what to load and when, is a discipline separate from prompt engineering.

2.3. Compound AI Systems vs. Monolithic Agents

The basic flaw in the early agent design has been the excessive dependency on a monolithic architecture – in other words, the assumption that one large language model (LLM) would act as the brain, data storage, and executor all at once. The described "one model does everything" approach is fragile because a sole model trying to keep state, plan tasks, and make API calls would be vulnerable to either hallucinations or cognitive overload based on its context load (Amar Gupta, 2025).

Contemporary engineering tends to follow the path of Compound AI Systems. Academia defines successful agents as not just "LLMs with database access" but as carefully designed systems made of separated and purpose-built components. A true compound system describes the architecture in several layers: a main reasoning "brain," a separate data layer that manages state in real time, explicit code executors for physical action, and micro-models for specific domains.

This separation exactly matches the V project architecture, which is divided into a separate routing engine, Semantic Gates, and task agents, thus placing it in a favourable position on the technical development pathway. The efficiency of a compound system does not come from one large neural network, but from deterministic interactions between these components where the main reasoning module processes inputs, uses isolated memory units for planning, and explicitly sends actions to the sandboxed executors.

However, the literature has one very important criterion – the realist criterion. Many agents are simply not needed since a predictable workflow and 80% or more pre-mapped paths of user actions make deterministic code much better. Compound AI Systems should be used when there is guaranteed unpredictability and tasks requiring dynamic routing across several data sources and decisions in real time.

3. Technology and Tools

The development process of V needed a technology stack created not just for raw computational power, but for strict deterministic control and operational transparency. Shifting away from the fragile, unpredictable nature of monolithic AI agents, this spine is built as a compound system where discrete tools execute highly specialized, isolated functions.

Python serves as the concrete foundation driving a custom event-driven orchestration layer via FastAPI that routes tasks efficiently without caging the system into rigid pipelines. Underpinning the reasoner, cutting-edge LLMs provided by companies such as OpenAI and Anthropic form the basis of the dual-LLM Planner and Synthesizer architecture.

To actively fight off the effects of context rot and database locking, memory management has been intentionally dichotomized in such a way that ChromaDB allows for seamless vector semantic queries, whereas SQLite enables a systematic and auditable long-term memory. Considering the autonomy of the system and how it performs complex loops on its own, the back-end operations of the system are continuously monitored using observability tools such as LangSmith and Weights & Biases. Lastly, the entire architecture is containerized and deployed using Docker and Uvicorn, and a reactive UI dashboard is created by Reflex.

The following table categorizes the core technologies and their specific functional roles within the research framework:

Category	Technology / Tools	Purpose and Functional Role
Language	Python	Serves as the primary programming language for backend scripts (main.py , llm_client.py , compaction.py), orchestrating tool calls, and managing system memory.
Core Framework	Custom Event-Driven Architecture / FastAPI	Provides the loop-based orchestration layer and routing gates to handle intent classification and execute the graph-based state machine without rigid pipelines.
Model Repository	OpenAI API / Anthropic (Claude)	Supplies the foundational Large Language Models used for the dual-LLM setup (Planner and Synthesizer) to handle reasoning, tool execution, and response generation.
Data Science	ChromaDB & SQLite	Powers the dual-database architecture. ChromaDB acts as the semantic vector search engine, while SQLite serves as the structured long-term memory (ROM) to guarantee data integrity.
Development	Visual Studio Code, Git, GitHub	Used for modular script development, strict version control of core prompts, and maintaining the repository for the agent's modular codebase.
Management	Weights & Biases (W&B), LangSmith	Utilized for logging execution traces, tracking model evaluation, observing tool calls, and managing agile project workflows.
Deployment	Docker, Uvicorn, Reflex	Handles the containerized local execution, backend serving (Uvicorn), and rendering the interactive UI (Reflex) for the desktop application.

Table 2: Technology and Tools

4. Software Product Requirements

4.1. Overview of Similar Products

Product Name	Core Technology	Strengths	Limitations
Letta	Hierarchical Memory Architecture (Core, Recall, Archival tiers)	Solves context rot by letting the agent autonomously manage working vs. long-term memory.	Requires heavy infrastructure setup (servers, DBs); overly complex for basic agent loops.
LangGraph	Graph-based State Machine (Directed graphs for state transitions)	Granular control to mix deterministic code with LLM routing; native LangSmith observability.	Very low-level abstraction requires writing extensive boilerplate code and rigid node definitions.
CrewAI	Role-Based Multi-Agent Delegation (Sequential/Hierarchical tasking)	Highly intuitive for splitting complex tasks into sub-tasks assigned to specialized personas.	Highly susceptible to compounding errors if intermediate agent outputs are not strictly validated.
OpenHands	Sandboxed Autonomous Execution (Docker-based terminal/browser access)	Safely executes live terminal commands and file edits, replicating real human engineering workflows.	Architecturally heavy and token-intensive; specifically optimized for software engineering, not general use.

Table 3: Similar Products

The current ecosystem of compound AI systems introduces distinct engineering trade-offs. Solutions like Letta mitigate context challenges but have considerable backend infrastructure requirements, while LangGraph gives complete routing freedom but is slow in development. The CrewAI tool allows orchestrating multiple

agents in an operation but suffers from poor error handling, while the OpenHands system ensures exact execution isolation but is deeply intertwined with the slow software engineering process.

Such an environment sheds light on the importance of Project V design as a natural evolution of existing solutions. By taking the structured memory compaction principle of Letta, the concepts of LangGraph, and secure local execution into a lightweight, local application, V agent closes the gap effectively. In addition to that, separating the Planner and Synthesizer while harnessing rigid database management (SQLite/ChromaDB) allows V to stay stateful, persistent, and transparent, with deterministic execution without crashing under the weight of enterprise-grade bloat.

4.2. User Stories and Use Case Diagram

To ensure the project's main goals align with real-world pain points, the following user stories represent the primary stakeholders within the AI-centric environments.

- 1. User Story 1:** As a normal user, typically a student or common adult, I want to freely chat and explore complex, long-running topics with AI without it forgetting earlier details or burning through my tokens, so I can trust it retains my context accurately over time.
- 2. User Story 2:** As a developer, I want a system that can simultaneously track, update, and act on my tasks without having me, the user, manually type out or perform actions more than talking. Therefore, I can spend my time on more meaningful tasks at hand.
- 3. User Story 3:** As a data-conscious user, I want a clear and easy-to-follow reactive dashboard that logs every AI's internal reasoning, active tool calls, and memory state to confidently trust that the system never makes up a task or facts.

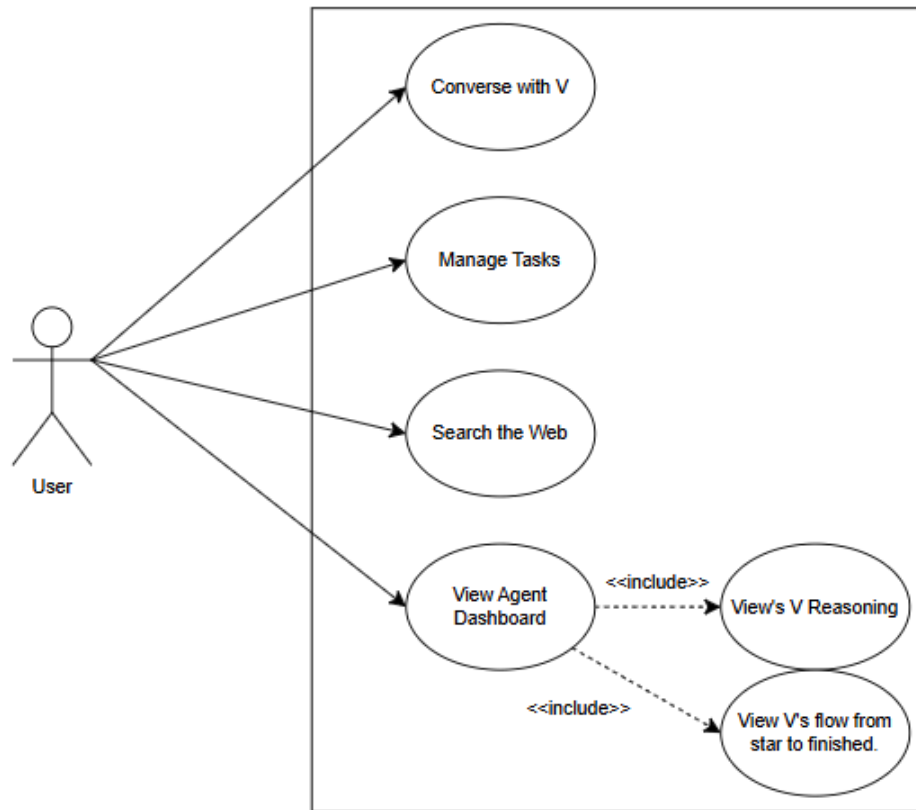


Figure 1: Use case diagram for V

In conclusion, these user stories and the provided use case diagram construct a strict, utility-driven foundation for Project V. Instead of relying on opaque AI behaviours, V itself is created to resolve real-world pain points:

Persistent Context: Delivering a token-efficient, long-term memory for casual interactions so users do not lose their track.

Hands-Free Automation: Providing autonomous task management for users who want to do more meaningful work through input rather than manual actions.

Absolute Observability: Enabling a reactive dashboard to audit the AI's internal reasoning, active tool calls and its memory state at all times.

The use case diagram translates these different users' needs into a cohesive, zero-trust engineering framework. This ensures that each action or stage is backed by a viewable and verifiable logic with clear system boundaries, as a result, eliminating the risk of unmonitored hallucinations effectively.

4.3. System Flow and Specifications

4.3.1. Use Case Specifications

This section provides a detailed description of the operational logic for the functionalities illustrated in the system's Use Case diagram.

1. Conver with V:

Field	Description
Actor	User
Description	The user interacts with the AI agent "V" through natural language to ask questions, request assistance, or initiate workflows.
Preconditions	The user has launched the desktop application and accessed the chat interface.
Post-conditions	V provides a contextually relevant response, executes underlying tools if needed, and logs the interaction into the memory buffer.

Table 4: Conver with V

Basic Flow:

- The user inputs a natural language prompt into the chat UI.
- The system passes the input through the Semantic Gate to analyze intent and retrieve relevant context from RAM/ROM.
- The orchestrator LLM generates a response based on the grounded context.
- The system displays V's response on the UI.
- The system logs the interaction into the `ram_window` buffer.

2. Manage Tasks (CRUD):

Field	Description
Actor	User, V (System Agent)

Description	The user (or the agent autonomously) creates, reads, updates, or deletes tasks, including managing deadlines and priorities.
Preconditions	The system is active, and the SQLite tasks database is accessible.
Post-conditions	The tasks database is updated successfully, and the system reflects the new state on the frontend.

Table 5: Manage Tasks (CRUD)

Basic flow:

- The user requests a task modification via chat (e.g., "Add a high-priority meeting for tomorrow at 2 PM").
- V's intent classifier extracts the action (Create) and metadata (deadline, priority).
- If the requested action is destructive (e.g., Delete/Drop), the system pauses and prompts the user for Human-in-the-Loop approval.
- Upon approval (or directly for non-destructive actions), V executes the corresponding SQL command.
- The system confirms the successful database operation to the user via the chat UI.

3. Search the Web:

Field	Description
Actor	User, V (System Agent)
Description	The agent dynamically queries the internet to retrieve up-to-date, real-world information to resolve user queries that fall outside its static training data or local memory.
Preconditions	The user submits a prompt requiring current factual knowledge, and the system has an active internet connection.

Post-conditions	V successfully synthesizes external data into a factual, grounded response.
------------------------	---

Table 6: Search the Web

Basic flow:

- The user asks a question requiring external or real-time data.
- The orchestrator LLM identifies the knowledge gap and calls the designated web search tool from the Tool Registry.
- The system executes the search and retrieves the top relevant results.
- V synthesizes the retrieved data, applies factual validation guardrails, and formats the output.
- The system displays the final, grounded answer to the user.

4. View Agent Dashboard:

Field	Description
Actor	User
Description	The user views the internal state of the agent, including active tasks, approaching deadlines, and the status of the RAM/ROM memory layers.
Preconditions	The user has accessed the V desktop application.
Post-conditions	The user obtains a comprehensive overview of the system's operational status and pending task queue.

Table 7: View Agent Dashboard

Basic flow:

- The user selects the "Dashboard" view from the main application menu.
- The system queries the backend to retrieve current data from the tasks and rom_memory SQLite tables.
- The system aggregates the data and prepares visualizations.
- The system displays the structured overview, highlighting urgent deadlines and system health metrics.

4.3.2. Activity Diagram

The Activity Diagram below illustrates the workflow of the autonomous desktop agent V from start to finish. The specific steps include:

1. **Start:** The user submits a natural language request via the desktop chat interface.
2. **Check Memory Capacity (RAM Window):** The system evaluates the active conversational buffer (`ram_window.py`) against the context limit.
 - **If Threshold Exceeded:** Triggers the compaction engine (`compaction.py`) to summarize context and commit structured records into the SQLite ROM database before proceeding.
 - **If Within Limit:** Passes the conversation context directly to the intent classifier.
3. **Classify Intent & Route Task:** The orchestrator (`llm_client.py`) analyzes the query to determine if external tools are required.
 - **If Direct Query:** Generates a response immediately using active RAM context.
 - **If Tool Required:** Routes the request to the designated module in the Tool Registry (e.g., Task CRUD, Web Search).
4. **Evaluate Security Gate (Human-in-the-Loop):** The system evaluates whether the requested tool action is destructive (e.g., deleting a record or dropping a table).
 - **If Destructive:** Pauses execution and prompts the user for explicit confirmation.
 - **If Approved:** Proceeds to tool execution.
 - **If Rejected:** Aborts the operation and returns a cancellation message to the user.
 - **If Non-Destructive:** Bypasses human confirmation and executes the tool directly.
5. **Execute Tool & Gather Observations:** The system runs the corresponding script, performing operations against the database or web APIs, and captures the raw observations.
6. **Synthesize & Validate Output:** The LLM processes the observations through factual guardrails to eliminate hallucinations and build a grounded final output.
7. **Update Memory & Render UI:** The system appends the interaction to the `ram_window` buffer, updates the frontend dashboard (Reflex UI), and displays the final response to the user.
8. **End Node:** The operational loop completes, returning the system to an idle state awaiting the next user input.

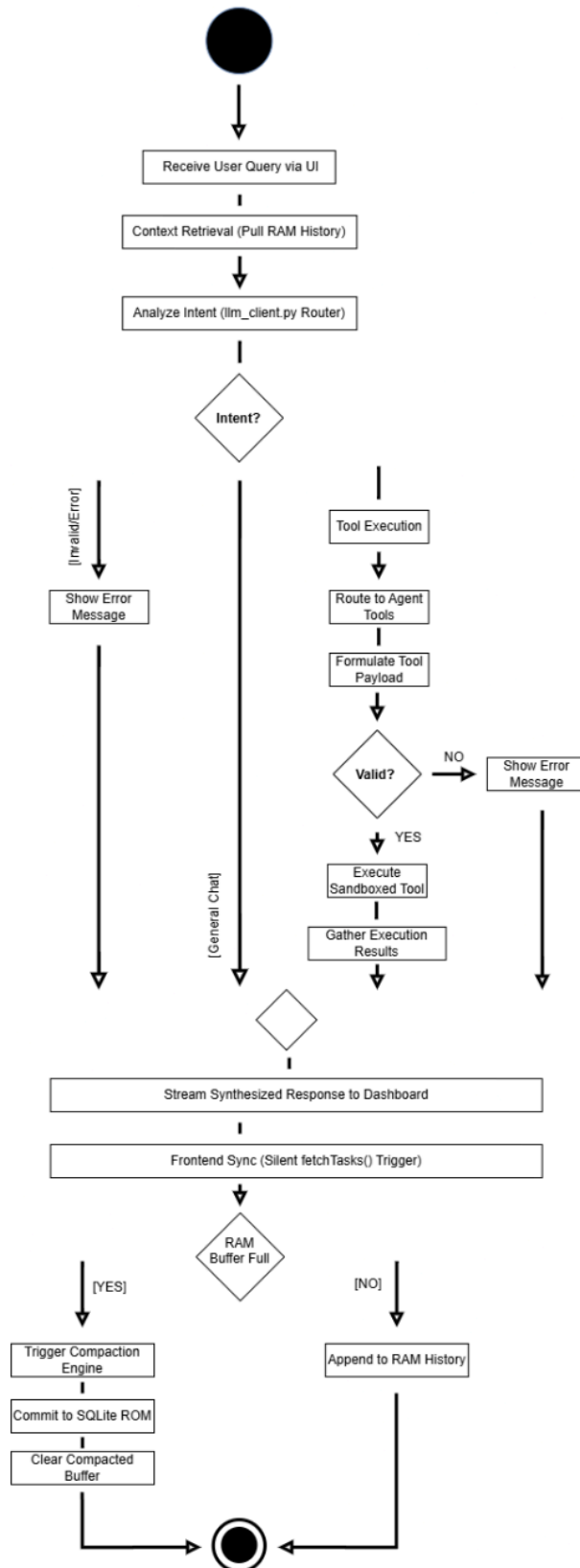


Figure 2: Activity Diagram

4.4. Entity Relationship Diagram (ERD)

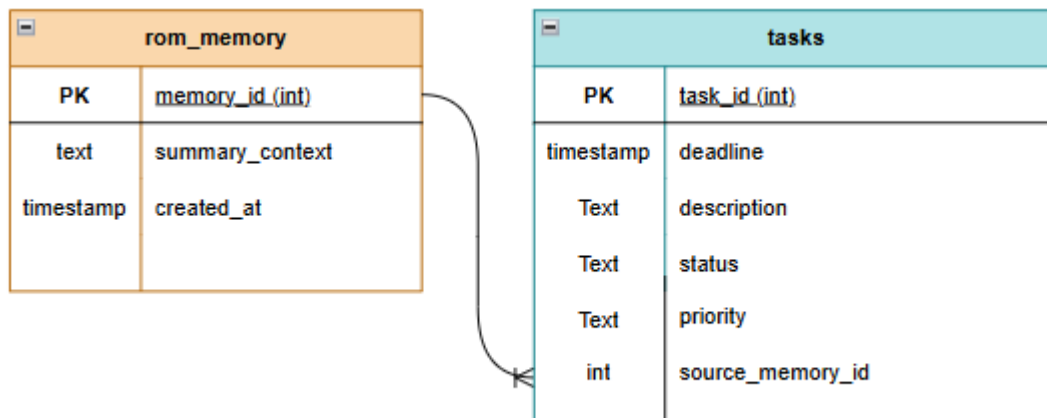


Figure 3: Entity relationship diagram of the V's system

Description of the Entity-Relationship Diagram (ERD):

The data model consists of two main context entities, which give context integrity and traceability of actions taken:

- **rom_memory:** Holds temporary conversation summaries/chunks of context.
- **tasks:** Holds tasks with due dates and priority, which has many-2-one relationship to **rom_memory** (more than one task can be created from 1 memory chunk).

Uses: This avoids context rot as everything the agent did has a breadcrumb trail that leads back to the original conversation that got stored in the agent's long-term memory.

4.5. Sitemap

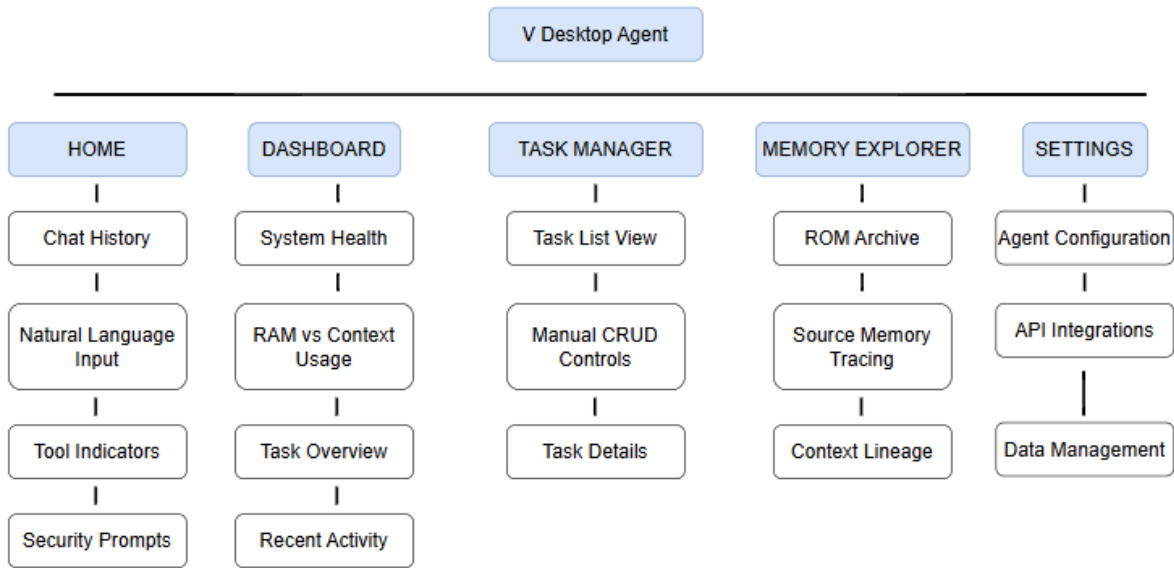


Figure 4: Sitemap of the AI Sentinel graphical user interface.

The sitemap provides the GUI architecture for the V desktop agent. The app is designed using five main modules to allow easy navigation and to distinguish the functional blocks:

- **Home:** Main conversational interface to communicate with the agent, track usage of tools and confirm or deny human-in-the-loop security prompts.
- **Dashboard:** Offers a general overview of system health, RAM consumption and recent activities of the agent.
- **Task Manager:** Interface for managing the tasks database that allows users to see deadlines and execute CRUD actions on the tasks database outside of chat.
- **Memory Explorer:** Access to the rom_memory of the agent that allows checking task summaries and finding a task lineage to the initial conversation.
- **Settings:** Manages system configuration such as API keys, database backups and strictness of LLM guardrails.

Such hierarchy makes switching between regular conversation and management of agent's memory/database parameters easy.

5. Software Development Methodologies

5.1. Waterfall Model

It is a linear sequential approach for software development. According to the Waterfall Model, there is a sequential process which must be strictly followed. In the process, each of the stages (Requirement, Design, Implementation, Testing, and Maintenance) should be completed and approved before proceeding to the next stage. One of the main features of the Waterfall Model is its rigidity and heavy reliance on documentation.

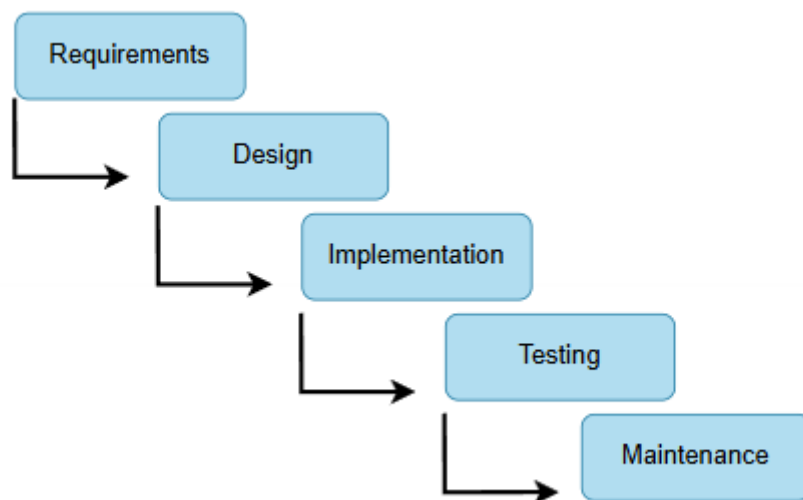


Figure 5: Waterfall Model

Advantages: Linear structure makes the model easy to understand, manage, and track. Clear milestones and strict requirements to complete documentation at each stage make sure that all aspects of the system design are well documented (Kalso, Reed, 2020).

Disadvantages: The model is too rigid and has poor adaptability. Due to the fact that the testing takes place later in the lifecycle, discovery of some serious mistakes in architecture and requirements can lead to fatal results and high cost of fixing the issue.

Applicability: The Waterfall Model is absolutely not appropriate for V. The construction of the AI agent with two-layer memory (RAM/ROM) and context management requires experiments and refinements constantly.

5.2. Spiral Model

This evolutionary software process puts together the iterative prototyping and controlled, systematic nature of the waterfall model. It is a risk-driven flow where the project is divided into many spirals, with each spiral pinpointing the identification and resolution of risks before diving into the next stage.

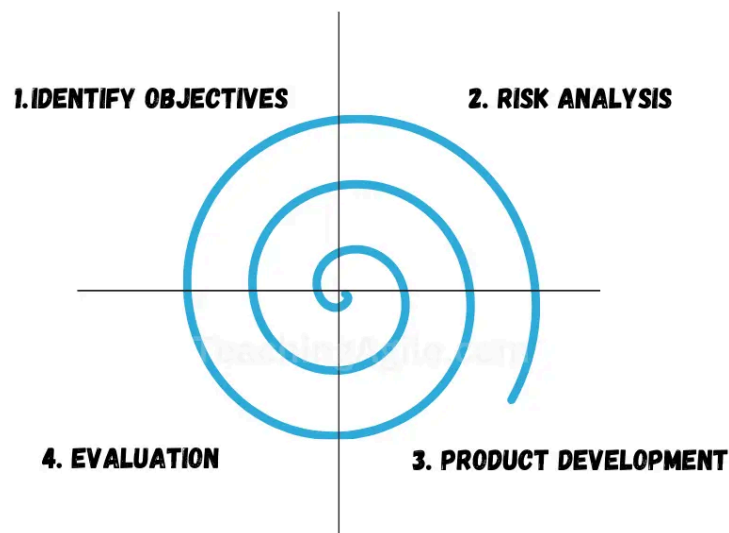


Figure 6: Spiral Model

Advantages: The model offers a better risk management process and therefore is the most suitable for big-budget, mission-critical, or any other complex project where the risks must be managed properly.

Disadvantages: It is a model that requires high complexity and overhead. It also requires risk assessment skills, which are costly and time-consuming, especially for small projects (Cameron Hashemi-Pour, 2025).

Applicability: While in nature, it proved to be a good model, the complexity and overhead associated with it are not considered at once, especially when being prioritized in small projects such as the current one where the focus is on development of specific MTL architecture.

5.3. Rapid Application Development (RAD / Prototyping)

Rapid Application Development (RAD) is an adaptive software development approach where quick prototyping and iterations take precedence over lengthy up-front planning and documentation. The key focus is on the speedy creation of a working prototype, its review and feedback from stakeholders, as well as design modification and iteration till the final product is reached.

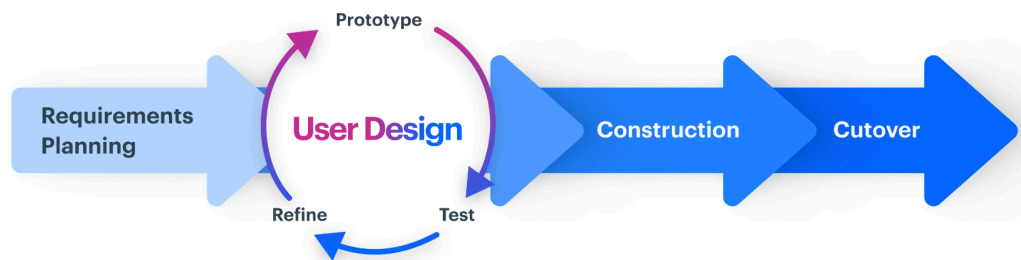


Figure 7: Rapid Application Development Model

Advantages: RAD allows great flexibility and fast development of functional parts (Cole Stryker, 2026). Focusing on prototypes makes it possible to quickly see how the system works and adjust the requirements based on empirical data rather than on predictions.

Disadvantages: Rapid development and prototyping may produce architecture-related problems if not appropriately controlled, which would result in the production of a system that is superficially polished but lacks scalability and maintainability.

Applicability: RAD fits very well for developing user-facing features of V—this can include the design of the user interface of Reflex or fine-tuning of natural language prompts. Yet, RAD is insufficient as an overall methodology for developing the agent; the key technical feature—the two-way synchronization of memory (RAM buffer and SQLite ROM)—needs a more planned technical approach.

5.4. Agile Methodology

Agile methodology implies an iterative and incremental approach which stresses the constant delivery of small and functional software increments. Unlike the traditional approach where the whole project is accomplished in one step, the development process is broken down into smaller and time-restricted cycles (sprints). The method takes into account changes and requires a lot of testing, evaluation, and adjustment within the entire project cycle (Artem Gurnov, 2026).

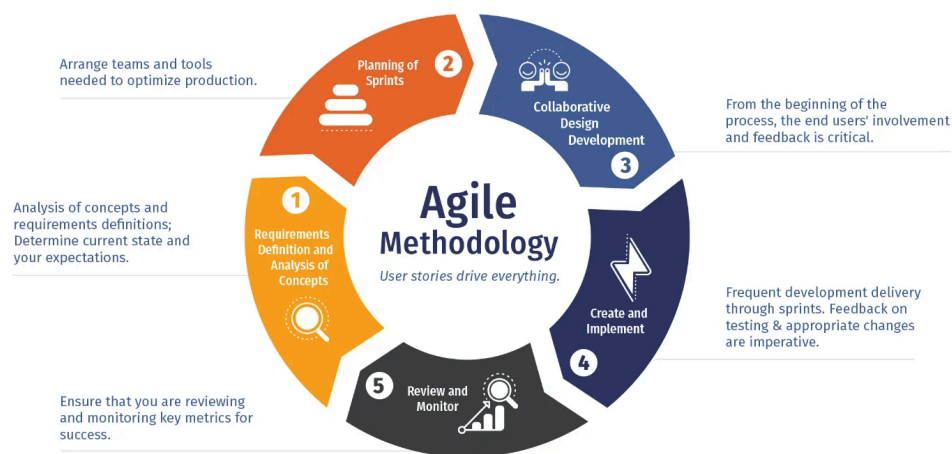


Figure 8: Agile Framework

Advantages: The Agile methodology provides excellent adaptability to changing requirements and technical issues. Regular assessment of the software after every short sprint guarantees constant progress and allows making corrections when the result of an experiment fails.

Disadvantages: Due to the flexibility of the Agile methodology, there might occur a problem with scope creep in case when the project limits and backlog of tasks are not strictly regulated. Besides, since the stress is put on producing working software, documentation might not receive enough attention.

Applicability: Agile is a suitable methodology for the creation of the V desktop agent. The structure of the sprints fits the timing of the graduation project; besides, it allows implementing complex features like semantic gates, memory compactors and tools running within the framework of iterative trial and error of building compound AI systems.

5.5. Selection of Methodology and Justification

After careful comparison and extensive research, process and comparative analysis of SDLC methodologies, a final verdict that a standalone traditional methodology cannot suffice to solve the different aspects of this project. which including structured database design and experimental AI integration. Thus, to facilitate the implementation of the V desktop agent, a Hybrid Agile Prototyping Methodology has been selected. This combination makes use of the agility of Agile to handle the back-end processes and memory management, along with the rapid development of the conversation interface using Prototyping.

Justification for the Selected Model:

1. Iterative System Development:

The project is divided into short-term development cycles (sprints), each focused on some particular goal like setting up RAM/ROM architecture, improving the context compaction engine, or trying out semantic routing gates. This way, we ensure constant progress in our work while keeping design flexible.

The project will consist of four logical sprints ensuring constant delivery and improvement of the system:

- **Sprint 1:** Core Architecture & Database Setup (SQLite schema creation for tasks and rom_memory, basic CRUD).
- **Sprint 2:** Memory Management & Logic (RAM buffer implementation, Context Compaction engine implementation, Semantic Gates implementation).
- **Sprint 3:** Tool Integration & Agentic Reasoning (Web Search Integration, Causal Tools Execution, Guardrails).
- **Sprint 4:** Application Development & UI Integration (Reflex frontend, Task Manager & Memory Explorer integration).

2. Ability to Accommodate Experimentation & Non-linear Workflow:

The process of developing Compound AI is non-linear. The use of agile methodology provides a possibility to have multiple cycles of testing, evaluation and refinement that are needed for prompt engineering, compaction threshold adjustment, token limit optimization, etc.

3. Ability to Adapt to Changing Requirements:

Constraints in the system are not known in advance. If new experiment results are obtained related to the latency of the LLM or its reasoning abilities, then the course of development could be changed (from monolithic prompts to modular agent tools).

4. Early Validation and Constant Feedback:

Every sprint includes testing phases that allow detecting problems with the system, such as memory hallucinations, infinite tool loop, DB bottleneck, etc. This helps to improve the system's reasoning capability in every sprint instead of debugging a God Agent at the last project stage.

5. Integration with Prototyping Approach for Frontend Development:

The prototype-based approach is used along with the agile one to quickly create and modify the interface for our system using the Reflex library. This allows quickly visualizing the state of our agent (e.g. Dashboard & Task Manager) and improving the user experience through continuous human-in-the-loop interaction and tool feedback.

6. Design and Implementation

6.1. Product Analysis and Design

- 6.1.1. Architecture Flow Design
- 6.1.2. Database Schema Design
- 6.1.3. System Interface Overview & GUI Design Process

6.2. Graphical Interface Illustrations

- 6.2.1. V Home
- 6.2.2. Dashboard
- 6.2.3. Task Manager
- 6.2.4. Memory Explorer
- 6.2.5. Setting & Configurations

6.3. Product Implementation

6.3.1. Database & Backend Integration

6.3.2. Agentic Logic & Memory Compaction

6.3.3. Tool Execution & API Integration

6.4. System Testing and Results

6.4.1. Evaluating Context Retention

6.4.2. Tool Execution Accuracy & Guardrail Testing

6.5. Product Evaluation

6.5.1. Strengths

6.5.2. Limitations and Challenges

7. Conclusion

7.1. Knowledge, Experience, and Insights

7.2. Achievement Assessment

7.3. Future Works

8. References

- [Why AI Agents Keep Failing in Production](#)
- [Understanding why AI Agents fail](#)
- [Context compaction for AI agents: a complete guide.](#)
- [Progressive Disclosure in AI Agents: How to Load Context Without Killing Output Quality](#)
- [Harness engineering and agent feedback: Exploring AI coding sensors](#)
- [What Is an Agent Execution Sandbox?](#)
- [The Complete Guide to AI Agent Architecture](#)
- [What Is Agile Methodology in Project Management?](#)
- [What is the spiral model and how is it used?](#)
- [What is rapid application development?](#)
- [Waterfall model](#)

9. Appendix
10. Declaration of AI Use